Module 12 - Threading Advanced

Table of Contents

- [ ] 1. `ThreadPoolExecutor` — The Modern Thread Pool
- [ ] 2. Thread-Safe `Queue` — The Right Way to Pass Data
- [ ] 3. Daemon Threads — Background Workers
- [ ] 4. Practical Pattern: Threaded Log Ingestion Pipeline
- [ ] 5. Summary and Next Steps

---

1. `ThreadPoolExecutor` — The Modern Thread Pool

Manually creating, starting, and joining threads works — but it becomes verbose when you're managing dozens of threads. `concurrent.futures.ThreadPoolExecutor` is the modern, cleaner approach built into the standard library.

It manages a **pool** of worker threads, automatically distributing tasks across them. You specify how many workers (threads) the pool should have, then submit tasks.

`map()` — the simple case

```
from concurrent.futures import ThreadPoolExecutor

with ThreadPoolExecutor(max_workers=10) as executor:
    results = list(executor.map(function, iterable))
# All threads finished when the 'with' block exits
```

`submit()` + `Future` — the flexible case

```
from concurrent.futures import ThreadPoolExecutor, as_completed

with ThreadPoolExecutor(max_workers=10) as executor:
    futures = {executor.submit(function, arg): arg for arg in items}

    for future in as_completed(futures):    # process results as they
arrive
        arg = futures[future]               # which item this future
belongs to
        result = future.result()            # get the return value
```

`map()` vs `submit()`

	`map()`	`submit()`
Result order	Preserves input order	Results arrive as completed
Error handling	Raises at iteration	Per-future exception handling
Best for	Simple, uniform tasks	Tasks with varying duration or error handling

In [ ]:
```
# Topic: ThreadPoolExecutor.map() — parallel log scan, clean syntax

import os
import time
from concurrent.futures import ThreadPoolExecutor


def get_host_event_summary(hostname):
    """Read one host log and return a summary dict."""
    counts = {"failed_logins": 0, "intrusions": 0, "port_scans": 0}

    with open(f"data/{hostname}.log", "r") as f:
        for line in f:
            if "SSH_LOGIN_FAILED" in line:   counts["failed_logins"] += 1
            if "INTRUSION_ATTEMPT" in line:  counts["intrusions"]    += 1
            if "PORT_SCAN" in line:          counts["port_scans"]    += 1

    return hostname, counts


hostnames = [f.replace(".log", "") for f in sorted(os.listdir("data")) if f.endswit

start = time.time()

# max_workers=10: one thread per host — optimal for 10 small IO tasks
with ThreadPoolExecutor(max_workers=10) as executor:
    results = list(executor.map(get_host_event_summary, hostnames))
    # executor.map preserves ORDER — results match the order of hostnames

elapsed = time.time() - start

print(f"Scanned {len(results)} hosts in {elapsed:.2f}s\n")
print(f"{'Host':<12} {'Failed Logins':>14} {'Intrusions':>12} {'Port Scans':>11}")
print("-" * 52)
for hostname, counts in results:
    print(f"{hostname:<12} {counts['failed_logins']:>14} {counts['intrusions']:>12}
```

In [ ]:
```
# Topic: submit() + as_completed() — process results as they arrive
# Useful when tasks have different durations — don't wait for the slowest

import os
import time
from concurrent.futures import ThreadPoolExecutor, as_completed
```

```
def check_host_for_brute_force(hostname):
    """Check if a host shows signs of brute force: 3+ failed logins."""
    failed = 0
    success_after_fail = False
    last_was_failed = False

    with open(f"data/{hostname}.log", "r") as f:
        for line in f:
            if "SSH_LOGIN_FAILED" in line:
                failed += 1
                last_was_failed = True
            elif "Brute force succeeded" in line:
                success_after_fail = True

    risk = "CRITICAL" if success_after_fail else ("HIGH" if failed >= 3 else "LOW")
    return {"hostname": hostname, "failed_logins": failed,
            "compromised": success_after_fail, "risk": risk}


hostnames = [f.replace(".log", "") for f in sorted(os.listdir("data")) if f.endswit

critical_hosts = []

with ThreadPoolExecutor(max_workers=10) as executor:
    # submit() returns a Future for each task
    futures = {executor.submit(check_host_for_brute_force, h): h for h in hostnames

    # as_completed() yields futures as they finish — fastest first
    for future in as_completed(futures):
        result = future.result()
        print(f"  [{result['risk']:8}] {result['hostname']} — "
              f"{result['failed_logins']} failed, compromised: {result['compromised
        if result["risk"] == "CRITICAL":
            critical_hosts.append(result["hostname"])

print(f"\nHosts requiring immediate response: {critical_hosts}")
```

🎯 Practice — ThreadPoolExecutor

Exercise 1 — Write

Using `ThreadPoolExecutor.map()`, write a function `extract_attacker_ips(hostname)` that returns the set of unique IPs that appear in `SSH_LOGIN_FAILED` events in `data/{hostname}.log`.

Run it across all 10 hosts. Combine all results into one master set `all_attacker_ips`. Print the total unique attacker IPs found across the infrastructure.

In [ ]:
```
# Your code here
import os
from concurrent.futures import ThreadPoolExecutor
```

▶ 💡 **SOLUTION**

---

2. Thread-Safe `Queue` — The Right Way to Pass Data

`queue.Queue` is a thread-safe data structure for passing items between threads. Unlike a plain list — which needs a `Lock` to be safe — `Queue` handles its own synchronization internally.

It implements the **producer-consumer pattern**:

- **Producer threads** put items onto the queue
- **Consumer threads** get items from the queue

Method	What it does
`q.put(item)`	Add an item (blocks if queue is full)
`q.get()`	Remove and return an item (blocks if queue is empty)
`q.task_done()`	Signal that a get()'d item has been processed
`q.join()`	Block until all items have been processed
`q.empty()`	True if the queue is empty

Why Queue instead of a list + Lock?

Using a `Queue` is cleaner: you don't need to write the locking logic yourself, and `q.join()` gives you a clean way to know when all work is done.

In [ ]:
```
# Topic: Queue — producer/consumer for log event aggregation
# Producer threads: read log files, put critical events on the queue
# Consumer thread: read from the queue, write to an incident report

import os
import queue
from threading import Thread

# Sentinel: a special value that signals "no more items"
SENTINEL = None

event_queue = queue.Queue()


def producer(hostname):
    """Read a log file and put critical events on the queue."""
    with open(f"data/{hostname}.log", "r") as f:
```

```
        for line in f:
            line = line.strip()
            if "INTRUSION_ATTEMPT" in line or "Brute force succeeded" in line:
                event_queue.put((hostname, line))   # put a tuple on the queue


def consumer(output_path, num_producers):
    """Read events from the queue and write them to a report file."""
    producers_done = 0

    with open(output_path, "w") as report:
        report.write("=== Critical Event Report ===\n\n")

        while True:
            item = event_queue.get()   # blocks until an item is available

            if item is SENTINEL:       # producer finished — count sentinels
                producers_done += 1
                if producers_done == num_producers:  # all producers done
                    break
                continue

            hostname, line = item
            report.write(f"[{hostname}] {line}\n")


hostnames = [f.replace(".log", "") for f in sorted(os.listdir("data")) if f.endswit

# Start consumer first — it waits for producers
consumer_thread = Thread(target=consumer, args=("data/critical_report.txt", len(hos
consumer_thread.start()

# Start producer threads
producer_threads = [Thread(target=producer, args=(h,)) for h in hostnames]
for t in producer_threads:
    t.start()

# Wait for all producers, then send one sentinel per producer to signal done
for t in producer_threads:
    t.join()
    event_queue.put(SENTINEL)   # one sentinel per finished producer

consumer_thread.join()          # wait for consumer to finish writing

# Read the result
with open("data/critical_report.txt", "r") as f:
    print(f.read())
```

🎯 Practice — Queue

Exercise 2 — Write

Use a `queue.Queue` to build a producer-consumer pipeline:

- **10 producer threads** — one per host — each reads its log file and puts `PORT_SCAN` event lines onto the queue
- **1 consumer thread** — reads from the queue and builds a dict `port_scan_by_host: {hostname: count}` (parse hostname from the log line)

After the pipeline finishes, print the dict.

In [ ]:
```
# Your code here
import os, queue
from threading import Thread
```

▶ 💡 **SOLUTION**

---

3. Daemon Threads — Background Workers

By default, Python waits for all threads to finish before the program exits, even if the main code has completed. This can cause your script to hang.

A **daemon thread** is a background thread that is automatically killed when the main program exits — it does not prevent shutdown.

```
t = Thread(target=monitor_function)
t.daemon = True     # mark as daemon before starting
t.start()           # runs in background — dies when main exits
```

When to use daemon threads in security tools

Use case	Daemon?	Reason
Worker thread processing a task list	No	You need the result — must complete
Background heartbeat monitor	Yes	Runs forever — should not block exit
Log tail watcher	Yes	Runs forever — should not block exit
Alert notification sender	No	Must finish sending before exiting

⚠️ Warning

Daemon threads are killed **immediately** — with no cleanup. Do not use daemon threads for tasks that write to files, databases, or send alerts unless you handle cleanup explicitly.

In [ ]:
```
# Topic: daemon thread — background heartbeat monitor
# In a real tool this would check connectivity to a SIEM or alert system

import time
from threading import Thread
```

```python
def heartbeat_monitor(interval_seconds):
    """Periodically check that the SIEM is reachable. Runs forever as a daemon."""
    check_count = 0
    while True:
        check_count += 1
        # In a real tool: try connecting to SIEM host:port
        print(f"  [HEARTBEAT #{check_count}] SIEM connectivity: OK")
        time.sleep(interval_seconds)


# Start heartbeat as a daemon — it will be killed when main exits
monitor = Thread(target=heartbeat_monitor, args=(0.5,))
monitor.daemon = True     # MUST set before start()
monitor.start()

print("Main: starting log analysis...")
time.sleep(1.8)           # simulate doing real work
print("Main: analysis complete — exiting")
# Daemon thread is automatically killed here — no hang
```

---

4. Practical Pattern: Threaded Log Ingestion Pipeline

This section combines everything — `ThreadPoolExecutor`, `Queue`, locks, and file writing — into a realistic security tool pattern.

The problem

You have 10 server log files. You need to:

1. Read all 10 in parallel (IO-bound — threading helps)
2. Filter for security-relevant events
3. Deduplicate attacker IPs
4. Write a structured report: counts by event type + attacker IP list

This is the kind of script a SOC analyst writes to process a morning log dump.

In [ ]:
```python
# Topic: complete log ingestion pipeline

import os
import json
import time
from concurrent.futures import ThreadPoolExecutor
from threading import Lock


# ── Shared state ──
aggregated = {
    "total_events":     0,
```

```
    "failed_logins":    0,
    "intrusions":       0,
    "port_scans":       0,
    "brute_force_wins": 0,
}
attacker_ips = set()
critical_lines = []
state_lock = Lock()


def ingest_host_log(hostname):
    """Process one host log — called by a thread pool worker."""
    local_counts = {k: 0 for k in aggregated}
    local_attackers = set()
    local_critical = []

    with open(f"data/{hostname}.log", "r") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue

            parts = line.split(" | ")
            source_ip = parts[2].strip() if len(parts) >= 3 else ""
            event_type = parts[3].strip() if len(parts) >= 4 else ""

            local_counts["total_events"] += 1

            if event_type == "SSH_LOGIN_FAILED":
                local_counts["failed_logins"] += 1
                local_attackers.add(source_ip)
            elif event_type == "INTRUSION_ATTEMPT":
                local_counts["intrusions"] += 1
                local_attackers.add(source_ip)
                local_critical.append(f"[{hostname}] {line}")
            elif event_type == "PORT_SCAN":
                local_counts["port_scans"] += 1
                local_attackers.add(source_ip)
            elif "Brute force succeeded" in line:
                local_counts["brute_force_wins"] += 1
                local_critical.append(f"[{hostname}] COMPROMISED: {line}")

    # Commit local results — hold lock briefly
    with state_lock:
        for key, value in local_counts.items():
            aggregated[key] += value
        attacker_ips.update(local_attackers)
        critical_lines.extend(local_critical)

    return hostname   # return value for progress tracking


hostnames = [f.replace(".log", "") for f in sorted(os.listdir("data")) if f.endswit

print(f"Ingesting logs for {len(hostnames)} hosts...")
start = time.time()
```

```
with ThreadPoolExecutor(max_workers=10) as executor:
    for completed_host in executor.map(ingest_host_log, hostnames):
        print(f"  [OK] {completed_host}")

elapsed = time.time() - start

# Build and write the final report
report = {
    "ingestion_time_seconds": round(elapsed, 3),
    "hosts_processed":        len(hostnames),
    "event_counts":           aggregated,
    "unique_attacker_ips":    sorted(list(attacker_ips)),
    "critical_events":        sorted(critical_lines),
}

with open("data/ingestion_report.json", "w") as f:
    json.dump(report, f, indent=2)

print(f"\nIngestion complete in {elapsed:.2f}s")
print(json.dumps(report, indent=2))
```

🎯 Practice — Full Pipeline

Exercise 3 — Write

Build a pipeline that:

1. Uses `ThreadPoolExecutor` to read all 10 host logs
2. Finds all `SSH_LOGIN_SUCCESS` events where the previous event from the same IP was `SSH_LOGIN_FAILED` (i.e., eventual success after failures — sign of brute force)
3. Collects the attacker IP, hostname, and timestamp for each such event
4. Writes results to `data/brute_force_successes.json`

Hint: in each host's log, look for lines where the IP matches an attacker (already seen in `SSH_LOGIN_FAILED`) and the event is `SSH_LOGIN_SUCCESS`.

In [ ]:
```
# Your code here
import os, json
from concurrent.futures import ThreadPoolExecutor
```

▶ 💡 **SOLUTION**

---

5. Summary and Next Steps

Advanced Threading — Quick Reference

Tool	Best for
`ThreadPoolExecutor(max_workers=N)`	Managing many threads cleanly
`executor.map(fn, items)`	Same function on many items, preserve order
`executor.submit()` + `as_completed()`	Variable-duration tasks, process as they finish
`queue.Queue`	Thread-safe data passing, producer-consumer patterns
`t.daemon = True`	Background monitors that should not block exit
Collect locally, write once with lock	Best practice for minimizing lock contention

Next Steps

- **02_Drills_Threading.ipynb** — 15 exercises across all threading topics
- After Threading: **Decorators** — adding functionality to functions without modifying them
- The Concurrent Security Scanner project combines threading with sockets — see the capstone notebook